PjBL: Duas Empresas, Duas Restrições - Deduplicação de Leituras de Sensores

Complexidade de Algoritmos

1. Apresentação e Contexto

Este trabalho tem como objetivo consolidar a análise de complexidade de tempo e de espaço através de um problema concreto: eliminar leituras duplicadas em um fluxo de dados de sensores, entregando duas soluções distintas para dois clientes fictícios com restrições opostas.

A Embarca Ltda. fabrica sensores embarcados (temperatura, umidade, etc.) com poucos bytes de RAM disponíveis para processamento local; a Datacenter S.A. recebe os dados já filtrados de milhares de sensores e precisa reprocessá-los em lote, o mais rápido possível, sem restrição relevante de memória. O mesmo problema, dois perfis de cliente antagônicos.

O trabalho é dividido em duas frentes obrigatórias: uma análise teórica ampla, cobrindo dez estratégias distintas (Seção 4.2), e uma implementação restrita a apenas duas delas. As oito estratégias não implementadas não são descartáveis, elas precisam ser analisadas com o mesmo rigor das duas implementadas, incluindo quatro estratégias deliberadamente exageradas (Seção 5), cujo papel é testar se o grupo sabe rejeitar uma solução tecnicamente válida, mas impraticável, com argumentos quantitativos.

> *Todas as dez estratégias da Seção 4.2 devem ser analisadas teoricamente por todos os grupos. Apenas duas devem ser efetivamente implementadas. A escolha das duas, e a qualidade da extrapolação das outras oito, valem nota tanto quanto o código em si.*

2. Objetivos de Aprendizagem

Ao concluir este trabalho, o grupo deve ser capaz de:

- Analisar formalmente a complexidade de tempo e de espaço de múltiplas estratégias para o mesmo problema, distinguindo espaço de pilha (recursão) de espaço de heap (alocação dinâmica);
- Comparar a previsão teórica (Big-O) com medições empíricas reais de tempo de execução e memória alocada;
- Tomar decisões de engenharia sob restrições concretas e conflitantes, justificando a escolha de um algoritmo em vez de outro com números, não apenas intuição;
- Reconhecer quando uma estratégia teoricamente correta é impraticável em um cenário real, e articular exatamente por quê e sob qual restrição específica ela falha;
- Instrumentar manualmente o programa para medir tempo de execução, profundidade de pilha e bytes alocados no heap, na ausência de um profiler pronto.

3. Formação dos Grupos

- Todos os integrantes devem ser capazes de explicar qualquer parte do trabalho entregue, incluindo a análise teórica das estratégias não implementadas. O professor pode arguir individualmente qualquer membro do grupo sobre qualquer item;
- O par de estratégias a ser implementado (Seção 4.3) é submetido à aprovação do professor antes do início da implementação, para garantir diversidade de escolhas entre os grupos da turma;
- Recomenda-se o uso da linguagem C para Embarcada e Python para a Datacenter.

4. Especificação do Projeto

4.1 Restrições Explícitas

As restrições abaixo são as mesmas para todos os grupos e não podem ser flexibilizadas na análise nem na implementação:

Elemento	Definição
Domínio dos dados	Leituras de sensor como inteiros de 16 bits, faixa fixa e conhecida: 0–65535
Tamanho de entrada	Testado em pelo menos três escalas: $n = 10^3$, 10^5 e 10^7
Cliente Embarca Ltda.	Memória extra limitada a 256 bytes, independentemente de n, além do próprio vetor de entrada já residente na memória do sensor
Cliente Datacenter S.A.	Sem limite de memória; orçamento de tempo: processar $n = 10^7$ em menos de 2 segundos na máquina de referência da disciplina
Correção exigida	Exata para ambos os clientes, exceto se a estratégia escolhida for probabilística, nesse caso, a taxa de falso positivo declarada deve ser ≤ 1%

4.2 Catálogo de Estratégias

As dez estratégias abaixo devem ser analisadas teoricamente por todos os grupos, com dedução formal de tempo e espaço (pilha e heap separados quando aplicável). As quatro últimas são propositalmente exageradas, seu papel é descrito na Seção 5.

#	Estratégia	Tempo	Espaço extra	Observação
1	Força bruta (comparar cada leitura com todas as já vistas)	$O(n^2)$	$O(1)$	Único candidato "seguro" para a Embarca sem conhecer o domínio dos dados
2	Ordenação + varredura de vizinhos	$O(n \log n)$	$O(1)$ a $O(n)$, depende do sort escolhido	Reabre a discussão in-place vs. auxiliar dentro do próprio projeto
3	Tabela hash / conjunto	$O(n)$ médio, $O(n^2)$ pior caso	$O(n)$	Bom caso médio, sem garantia — gancho para amortizada
4	Árvore binária balanceada (AVL / Red-Black)	$O(n \log n)$ garantido	$O(n)$	Contraponto determinístico ao hash
5	Bitmap / vetor de bits sobre o domínio conhecido	$O(n)$	8192 bytes fixos, independente de n	Só é viável porque o domínio (16 bits) é conhecido
6	Filtro de Bloom	$O(n)$	$O(m)$, quase constante, ajustável	Troca correção por espaço
7	Bogosort + varredura (embaralhar até ordenar)	$O(n \cdot n!)$ esperado	$O(1)$	Estratégia absurda
8	Enumeração recursiva de subconjuntos (power set)	$O(2^n)$	$O(2^n)$ heap + $O(n)$ de pilha	Estratégia absurda
9	Codificação unária em disco	O(soma dos valores)	~$O(1)$ em RAM, exponencial em disco	Estratégia absurda
10	Matriz de adjacência n×n de auditoria	$O(n^2)$	$O(n^2)$ desnecessário	Estratégia absurda

4.3 Regra de Implementação: dez avaliadas, duas implementadas

- O grupo entrega a análise teórica das dez estratégias: tempo, espaço, melhor/médio/pior caso, e sob qual restrição de qual cliente cada uma seria aceitável ou não;
- Dentre as dez, o grupo escolhe apenas duas para implementar de fato, com justificativa explícita de por que essas duas cobrem o espectro necessário para atender aos dois clientes;
- Para as oito estratégias não implementadas, o grupo precisa extrapolar: posicionar essas curvas no mesmo gráfico empírico das duas medidas, com base apenas na análise teórica;
- O par de estratégias implementado é submetido ao professor para aprovação prévia, garantindo diversidade de escolhas entre os grupos da turma.

5. Estratégias "Absurdas": Por que Elas Importam

As quatro estratégias a seguir não são pegadinhas gratuitas: cada uma é tecnicamente uma solução válida ao problema, e cada uma viola exatamente um tipo de restrição, de um jeito diferente. O objetivo é testar se o grupo sabe rejeitar uma estratégia com argumento quantitativo, quanto tempo, quantos bytes, em qual recurso, em vez de apenas dizer "é ruim".

Estratégia	O que ela viola	O que ela ensina
7. Bogosort + varredura	Restrição de tempo de ambos os clientes, mesmo para n pequeno, o tempo esperado é astronomicamente maior que qualquer alternativa da tabela	Complexidade assintótica não é o único critério: uma solução "O(1) de espaço" pode ser inaceitável na prática. Contraste direto com Quick Sort/Merge Sort já estudados em sala
8. Enumeração de subconjuntos (power set)	Restrição de memória da Embarca e de tempo de ambos, cresce exponencialmente mesmo para dezenas de elementos	Explosão combinatória por resolver um problema mais geral do que o necessário; risco real de estouro de pilha (stack overflow), conecta diretamente com pilha × heap
9. Codificação unária em disco	Não viola a restrição de RAM da Embarca, viola a premissa de que "memória" é apenas RAM. O custo é transferido para disco/I/O, e pode ser gigantesco	Big-O de espaço precisa declarar qual recurso está sendo medido; otimizar um recurso às custas de outro, escondido, não é uma solução real
10. Matriz de adjacência n×n de auditoria	Restrição de memória da Embarca, sem necessidade, o problema original nunca pediu rastreabilidade de todas as comparações	Over-engineering: gastar memória "por segurança" sem que isso tenha sido pedido é tão errado quanto economizar memória demais quando ela sobra

6. Requisitos Técnicos de Implementação

- Implementação em linguagem a escolha da equipe para ambas as implementações.
- O código deve conter uma rotina de demonstração que executa as duas estratégias implementadas para os três tamanhos de entrada definidos na Seção 4.1 e imprime tempo e memória medidos;
- Entradas vazias ou já sem duplicatas não podem gerar erro ou comportamento indefinido;
- Caso uma das duas estratégias escolhidas seja probabilística (Filtro de Bloom), o código deve expor o parâmetro que controla a taxa de falso positivo e relatar a taxa observada empiricamente.

7. Estrutura de Entrega

A entrega deve ser um único arquivo compactado (.zip) ou um repositório Git, contendo:

```
projeto/
├── src/
│   ├── <estrategia_1>          (primeira estratégia implementada)
```

```
│   ├── <estrategia_2>          (segunda estratégia implementada)
│   └── main                    (rotina de demonstração / benchmarks)
├── docs/
│   └── analise_teorica.md      (análise das dez estratégias)
├── dados/
│   └── resultados_empiricos.csv (tempo e memória medidos por n)
└── relatorio.pdf               (recomendação final por cliente)
```

8. Casos de Teste Obrigatórios

- Corretude: casos pequenos e verificáveis manualmente ($n \leq 10$) para as duas estratégias implementadas, com e sem duplicatas presentes;
- Limite de memória: evidência instrumentada de que a versão destinada à Embarca não ultrapassa 256 bytes extras, para todos os n testados;
- Orçamento de tempo: evidência de que a versão destinada à Datacenter processa $n = 10^7$ em menos de 2 segundos na máquina de referência;
- Validação teórica das estratégias absurdas: para cada uma das quatro estratégias da Seção 5, um cálculo numérico explícito demonstrando por que ela é inviável para pelo menos um dos dois clientes;
- Coerência gráfica: as oito curvas extrapoladas devem ser compatíveis, em ordem de grandeza, com a análise teórica, divergências devem ser explicadas no relatório, não escondidas.

9. Rubrica de Avaliação

A nota final do trabalho é a soma ponderada do desempenho do grupo em cada um dos seis critérios abaixo, conforme o peso indicado. Os percentuais de cada nível são aplicados sobre o peso do respectivo critério.

Critério	Peso	Insuficiente (0–40%)	Regular (41–70%)	Bom (71–90%)	Excelente (91–100%)
1. Análise teórica das dez estratégias	**25%**	menos de 6 estratégias analisadas, ou sem dedução de tempo e espaço	todas as dez analisadas, mas com erros relevantes ou sem separar pilha de heap	todas as dez corretas, com dedução clara de tempo e espaço	além de corretas, situam cada estratégia frente às restrições dos dois clientes com precisão
2. Justificativa da escolha das duas implementadas	**10%**	escolha não justificada, ou par não cobre nenhum cliente adequadamente	justificativa superficial, cobre um cliente bem e o outro de forma frágil	justificativa consistente, par cobre bem os dois clientes	justificativa compara explicitamente contra alternativas descartadas do catálogo
3. Implementação e instrumentação	**20%**	não compila, ou instrumentação de tempo/memória ausente	compila e mede tempo, mas instrumentação de memória incompleta ou incorreta	instrumentação completa e correta de tempo e memória para as duas estratégias	instrumentação robusta, incluindo casos de borda, com código limpo e comentado
4. Dados empíricos e coerência teoria-prática	**20%**	dados ausentes, incompletos, ou não comparados com a previsão teórica	dados coletados para os três n, mas comparação com a teoria é superficial	dados completos e comparados corretamente, divergências explicadas	análise crítica das divergências, incluindo efeitos de ambiente (cache, SO)
5. Validação das estratégias absurdas e extrapolação	**15%**	estratégias absurdas descartadas sem cálculo numérico, ou extrapolação ausente	rejeição correta mas genérica; extrapolação incompleta ou pouco precisa	rejeição com cálculo numérico concreto; extrapolação coerente das oito não implementadas	identifica cenários hipotéticos em que alguma estratégia absurda deixaria de sê-lo
6. Relatório, recomendação final e comunicação	**10%**	relatório desorganizado, sem recomendação clara por cliente	recomendação presente, mas pouco fundamentada em números	relatório claro, recomendação por cliente bem fundamentada em dados	relatório de qualidade profissional, pronto para apresentar a um cliente real

10. Considerações Finais

As estratégias absurdas da Seção 5 não devem ser tratadas com medo nem com deboche, o objetivo não é rir delas, é rejeitá-las com o mesmo rigor quantitativo usado para aceitar as boas. Um grupo que descarta o Bogosort dizendo apenas "é lento" aprendeu menos do que um grupo que calcula, para n = 1000, quantas permutações seriam necessárias em média.

Em caso de dúvida sobre qualquer ponto deste enunciado, inclusive sobre como tratar um cenário não previsto aqui, a orientação é: conversar com o professor, tomar uma decisão razoável e documentá-la em docs/analise_teorica.md. Essa é, também, uma habilidade avaliada pelo Critério 6.